RÉSULTATS-BF

Dossier d'architecture d'une infrastructure résiliente et scalable, décrite en code avec Vagrant

Groupe 04 : KOBENDE Pêgdwendé Evrard, PORGO Abdoul Nassirou, OUEDRAOGO Armand Hendrix, THIOMBIANO Yempabou Romaric Johan

Université Joseph Ki-Zerbo, IBAM, M1 Informatique. Virtualisation et Cloud Computing, Dr SOMDA. 26 octobre 2026

Le code est disponible sur https://github.com/PorgoAbdoulNassirou/Projet-Virtualisation-Cloud. Notre groupe travaille sur le réseau `192.168.59.0/24`. Chaque section indique qui l'a rédigée, et chaque chiffre cité renvoie à un fichier de résultats conservé dans `tests/resultats/`.

1. Contexte et exigences (THIOMBIANO)

Chaque année, au moment de la publication des résultats d'examens, le site tombe quelques minutes après l'annonce. Le programme n'y est presque jamais pour rien : c'est l'infrastructure qui n'a pas été prévue pour encaisser un pic de consultations, ni pour continuer à fonctionner quand une machine lâche. RÉSULTATS-BF permet à un candidat de consulter son résultat à partir de son matricule. L'application elle-même est volontairement très simple (trois points d'entrée, moins de cent lignes). Ce qui compte dans ce projet, c'est l'infrastructure qui l'entoure et notre capacité à montrer, mesures à l'appui, qu'elle tient.

Deux exigences non fonctionnelles guident tous nos choix :

Exigence	Ce qu'elle impose	Ce que nous avons mis en place
Résilience : la panne d'une machine ne doit pas interrompre le service	Aucun point de défaillance unique sur le chemin d'une requête	Au moins deux serveurs applicatifs sans état derrière HAProxy, qui ne leur envoie du trafic que s'ils répondent sur `/sante` (R4). Deux répartiteurs qui se partagent une IP virtuelle par VRRP (E1). Une base répliquée en flux, avec une procédure de bascule écrite et chronométrée (R5, R6)
Scalabilité : absorber plus de trafic en ajoutant des machines, sans réécrire la configuration	Le nombre de serveurs doit être un simple paramètre, et tout ce qui en dépend doit être généré	La variable `NB_APP` dans `.env`. La table `NOEUDS` du `Vagrantfile` sert de source unique : HAProxy, keepalived, `/etc/hosts` et les cibles Prometheus en sont tous déduits (R3)

À cela s'ajoutent des exigences de qualité. La plateforme doit se reconstruire sans aucune intervention manuelle (`vagrant destroy -f && vagrant up`, R1). Le provisionnement doit pouvoir être rejoué sans rien changer (R2). Aucun mot de passe ne doit apparaître dans le dépôt (R7), et seul le répartiteur expose un port vers la machine hôte (R8). Enfin, nous avons voulu mesurer plutôt qu'affirmer : chaque résultat de ce dossier a été produit par un script du dossier `tests/`, que n'importe qui peut relancer.

2. Architecture (PORGO)

2.1 Vue d'ensemble

Le client (navigateur, `curl` ou `ab`) s'adresse à une IP virtuelle, `192.168.59.5`, portée par l'un des deux répartiteurs HAProxy : lb1 en temps normal, lb2 si lb1 tombe. Les deux machines se transmettent cette adresse grâce au protocole VRRP, géré par keepalived. Le répartiteur distribue les requêtes à tour de rôle entre les serveurs applicatifs, qui lisent la base PostgreSQL primaire bdd1. Celle-ci est répliquée en continu vers bdd2. Enfin, la machine obs collecte les métriques de toutes les autres (extension E2). Le schéma complet figure dans le README du dépôt.

Machine	Adresse IP	RAM	Rôle	Statut
IP virtuelle	192.168.59.5	sans objet	Adresse du service, portée par lb1 ou lb2	E1
lb1	192.168.59.11	512 Mo	HAProxy 2.8 et keepalived, actif (priorité 150)	obligatoire
lb2	192.168.59.12	512 Mo	HAProxy 2.8 et keepalived, en attente (priorité 100)	E1
app1 à appN	192.168.59.21 à .2N	512 Mo	Service HTTP sans état, port 3000	obligatoire
bdd1	192.168.59.31	768 Mo	PostgreSQL primaire	obligatoire
bdd2	192.168.59.32	768 Mo	Réplique en flux, promouvable	obligatoire
obs	192.168.59.41	768 Mo	Prometheus et Grafana	E2

Le socle obligatoire représente 5 machines et 3 072 Mo de mémoire. Avec les deux extensions, on arrive à 7 machines et 4 352 Mo.

2.2 Plan d'adressage

Depuis la version 6.1.28, VirtualBox n'accepte par défaut les réseaux privés hôte que dans la plage `192.168.56.0/21`. Notre groupe utilise donc `192.168.59.0/24` (55 + 4), sans toucher au fichier `/etc/vbox/networks.conf`, ce qui aurait rendu le dépôt dépendant de la configuration de chaque poste. Le dernier octet indique le rôle de la machine : `.1x` pour la répartition, `.2x` pour les applications, `.3x` pour les données et `.4x` pour la supervision, l'adresse `.5` étant réservée à l'IP virtuelle. Pour cette raison, `NB_APP` est limité à 9 serveurs (de `.21` à `.29`), afin de ne jamais empiéter sur les bases.

2.3 Une seule source de vérité (R3)

La table `NOEUDS` du `Vagrantfile` est la seule description de l'infrastructure. À chaque commande `vagrant`, elle est écrite dans `config/plateforme.json` (et dans `config/noeuds.yml`). Les machines virtuelles lisent ce fichier dans `/vagrant/config/` par l'intermédiaire du script `scripts/plateforme.py`. C'est de lui que sont déduits la liste des serveurs dans HAProxy, la configuration de keepalived (pairs, interface, IP virtuelle), le fichier `/etc/hosts` de chaque machine et les cibles de Prometheus. Aucune adresse n'est écrite en dur dans un script ou dans une configuration.

2.4 Cloisonnement du réseau (R8)

Tout le trafic interne passe par le réseau privé `192.168.59.0/24`. Le seul port redirigé vers l'hôte est le port 8080, qui mène au port 80 de lb1, et il n'est ouvert que sur l'adresse locale `127.0.0.1` : il n'est donc pas visible depuis le réseau de la salle. Les autres ports redirigés (2222, 2200, etc.) sont ceux que Vagrant ajoute pour son propre accès SSH. La page de statistiques de HAProxy (port 8404) n'écoute que sur l'adresse privée du répartiteur.

3. Justification des choix

3.1 Répartition de charge avec HAProxy (PORGO)

Besoin	Ce que fait HAProxy	Pourquoi pas Nginx
N'envoyer du trafic qu'aux serveurs en bonne santé (R4)	Une sonde active interroge `GET /sante` chaque seconde (`option httpchk`, `http-check expect status 200`)	La version libre de Nginx ne fait que des contrôles passifs : elle découvre la panne sur la requête d'un utilisateur. Les sondes actives sont réservées à Nginx Plus
Ne pas montrer d'erreur quand un serveur tombe (V2)	`retry-on conn-failure empty-response response-timeout` et `option redispatch` rejouent la requête sur un	Possible avec `proxy_next_upstream`, mais sans sonde active

Besoin	Ce que fait HAProxy	Pourquoi pas Nginx
	autre serveur	
Ajouter des serveurs sans coupure (R3)	`systemctl reload` démarre un nouveau processus et garde les connexions en cours	Équivalent
Observer la répartition	En-tête `X-Serveur`, page de statistiques, exportateur Prometheus intégré au paquet	Statistiques limitées dans la version libre

Pour le délai d'éviction (R4), la sonde passe chaque seconde (`inter 1s`) et un serveur est écarté après deux échecs consécutifs (`fall 2`). HAProxy limite la connexion de la sonde à 1 seconde (`timeout connect`) et la lecture de sa réponse à 3 secondes (`timeout check`). Pour une panne franche, comme une machine éteinte, un service arrêté ou une connexion refusée, le pire cas est donc d'environ 3 secondes : une seconde d'attente avant la sonde, puis deux sondes d'une seconde. Nous avons mesuré 1,04 s avec une application de test, puis 1,39 s et 1,28 s avec l'application finale (section 6). Le réglage `rise 2` évite de réintégrer un serveur instable sur un seul succès.

Le délai de lecture était au départ d'une seconde. Nous l'avons porté à 3 secondes le 23 octobre, après un test d'intégration de la bascule de la base. Une fois bdd1 éteinte, la bibliothèque libpq essaie d'abord bdd1 avant de passer à bdd2, si bien que `/sante` répond 200 en 0,6 à 2,1 secondes. Avec une seconde de délai, la sonde déclarait hors service des serveurs qui fonctionnaient (état `L7TOUT`), et le service restait en 503 alors que la bascule avait réussi. Ce réglage a un coût : un serveur figé, qui accepte la connexion mais ne répond plus, serait écarté en 7 secondes environ dans le pire cas, donc au-delà des 5 secondes demandées. Ce cas ne fait pas partie des scénarios V2 et V4, et le rejeu sur `response-timeout` protège quand même les requêtes des utilisateurs. Le détour par la machine éteinte disparaît d'ailleurs dès que bdd1 est remise en service comme réplique (section 5.3) : `/sante` répond alors en 0,33 s.

La configuration de HAProxy est générée par `scripts/generer_config_lb.py`, puis vérifiée par `haproxy -c` avant d'être installée. Une configuration incorrecte arrête donc le provisionnement au lieu de casser un répartiteur en service.

3.2 Une application sans état qui suit la bascule de la base (KOBENDE)

Le service (`app/app.py`, 87 lignes) n'utilise que la bibliothèque standard de Python (`ThreadingHTTPServer`) et `psycopg2`, installé avec apt (`python3-psycopg2`). Nous n'utilisons pas `pip`, ce qui évite des téléchargements imprévisibles pendant le provisionnement.

Besoin	Choix dans l'application	Raison
N'importe quel serveur doit pouvoir répondre (R3, V2)	Aucun état local : ni session, ni fichier, ni compteur. Une connexion à la base par requête, toujours refermée	HAProxy peut envoyer ou rejouer chaque requête sur n'importe quel serveur. Ajouter ou perdre un serveur ne fait perdre aucune donnée
Survivre à la bascule de la base sans rien modifier (V4)	Chaîne de connexion `host=<bdd1>,<bdd2>` avec `target_session_attrs=read-write`	libpq essaie les hôtes dans l'ordre et ne garde que celui qui accepte les écritures. Avant la bascule, c'est bdd1. Quand bdd1 est éteinte, bdd2 encore réplique est refusée et `/sante` renvoie 503. Après `pg_promote()`, la même chaîne trouve bdd2 et `/sante` repasse à 200, sans redémarrer l'application
La sonde de HAProxy doit dire la vérité (R4)	`/sante` exécute réellement `SELECT 1` et ne renvoie 200 que si la requête réussit	Un serveur qui a perdu sa base sort de la répartition au lieu de renvoyer des erreurs
Répondre vite même si la base ne répond plus	`connect_timeout=2` et `statement_timeout=2000`	Une requête n'attend jamais indéfiniment. Sans base, le 503 arrive en 2,1 s
Ne jamais afficher d'erreur brute	Toute erreur `psycopg2.Error` devient une réponse JSON 503, et la requête	L'utilisateur reçoit une erreur propre, et le matricule ne peut pas servir à

Besoin	Choix dans l'application	Raison
	SQL est paramétrée	une injection SQL
Démarrer avant la base	La connexion n'est ouverte qu'au moment d'une requête	Le service démarre même si bdd1 n'est pas prête, répond 503, puis 200 dès que la base est disponible

Le script `scripts/app.sh` copie `app.py` dans `/opt/resultats-bf/`, pour ne jamais exécuter le programme depuis le dossier partagé `/vagrant`. Il construit le fichier de configuration `/etc/resultats-bf/app.env` à partir des adresses de bdd1 et bdd2 lues dans la source de vérité et du mot de passe `PG_MDP_APP` fourni par le `.env`. Ce fichier appartient à root et n'est lisible que par lui (mode 600). Le service systemd `resultats-app` tourne sous un utilisateur système dédié, avec `ProtectSystem=strict`, et `Restart=always` le relance au bout d'une seconde s'il s'arrête. Chaque fichier passe par la fonction `installer_si_change`, si bien que le service n'est redémarré que lorsque l'un d'eux a réellement changé (R2).

3.3 PostgreSQL, réplication en flux et perte de données possible (OUEDRAOGO)

Besoin	Choix	Raison
Un volume de données suffisant pour les mesures	Une table `resultats(matricule, nom, serie, moyenne, admis)` remplie de 50 000 lignes par `generate_series`, avec des matricules de `BF000001` à `BF050000` et des moyennes calculées de façon déterministe entre 0 et 20	On retrouve exactement les mêmes données à chaque reconstruction (R1). L'insertion utilise `ON CONFLICT DO NOTHING`, ce qui la rend rejouable (R2)
Garder une deuxième copie à jour (R5)	Réplication physique en flux, initialisée par `pg_basebackup -R -X stream` depuis bdd1	C'est le mécanisme intégré à PostgreSQL 16, sans logiciel en plus. bdd2 est une copie exacte que l'on peut promouvoir avec `pg_promote()`
Réplication synchrone ou asynchrone	Asynchrone, le réglage par défaut	Une écriture n'attend pas bdd2, donc bdd1 reste disponible même si bdd2 tombe. En contrepartie, les dernières transactions pas encore transmises seraient perdues si bdd1 tombait. Dans nos mesures, une écriture est lisible sur la réplique en moins d'une seconde. Comme notre application ne fait que lire des données chargées à l'installation, aucune donnée ne peut être perdue en pratique
Sécuriser les accès	`pg_hba.conf` généré : l'utilisateur `app_resultats` (lecture seule) et l'utilisateur `replicateur` ne sont acceptés que depuis `192.168.59.0/24`, avec `scram-sha-256`. Les mots de passe arrivent par l'environnement	Chaque compte n'a que les droits nécessaires, aucun secret n'est dans le dépôt (R7) et aucun accès n'est possible depuis l'extérieur du réseau privé (R8)
Ne pas abîmer les données	Données dans `/var/lib/postgresql/16/main`, jamais dans `/vagrant`	Le dossier partagé de VirtualBox ne respecte ni les verrous ni `fsync`, comme le rappelle l'énoncé
Ne jamais écraser une base à jour	`replique.sh` ne recopie bdd1 que si bdd2 n'a pas encore la base, et `bdd.sh` n'écrit rien sur une réplique	Le provisionnement reste rejouable même après une bascule (section 5.3)

3.4 Doubler le répartiteur avec VRRP et keepalived (PORGO)

Un répartiteur unique serait lui-même un point de défaillance. lb1 et lb2 ont donc la même configuration HAProxy, et keepalived (version 2.2.8) leur fait partager l'IP virtuelle `192.168.59.5` par VRRP.

Les priorités sont de 150 pour lb1 et de 100 pour lb2, et elles sont déduites de l'ordre de la table. Le répartiteur actif envoie une annonce chaque seconde (`advert_int 1`). Si lb2 ne reçoit plus d'annonce de lb1 pendant environ trois secondes, il prend l'adresse et prévient le réseau par un ARP gratuit, pour que l'hôte mette à jour son cache. keepalived surveille aussi HAProxy (`vrrp_script` avec `systemctl is-active haproxy`) : une machine dont HAProxy est arrêté abandonne l'adresse, même si elle reste allumée. Les annonces sont envoyées en unicast plutôt qu'en multicast, pour ne pas dépendre du comportement du réseau hôte de VirtualBox. Les deux machines démarrent en état `BACKUP` avec `preempt_delay 10` : quand lb1 revient, il attend 10 secondes, le temps que HAProxy soit prêt, avant de reprendre l'adresse. Le mot de passe VRRP est lu dans `.env` (keepalived n'accepte pas plus de 8 caractères) et le fichier qui le contient est protégé en mode 600.

C'est bien l'IP virtuelle, et non `localhost:8080`, qui constitue l'adresse du service : la redirection de port de Vagrant passe par la carte réseau NAT de lb1 et disparaît avec lui.

3.5 Supervision avec Prometheus et Grafana, extension E2 (THIOMBIANO)

Le circuit des métriques est le suivant. Sur chaque machine, `node_exporter` publie sur le port 9100 l'état du processeur, de la mémoire et le simple fait que la machine répond. HAProxy publie de son côté, sur le port 8404, le débit et l'état de chaque serveur applicatif. Prometheus, installé sur obs, interroge toutes ces cibles toutes les 5 secondes et conserve les séries. Grafana, sur la même machine, les affiche dans un tableau de bord intitulé « RÉSULTATS-BF, plateforme ».

Besoin	Choix	Raison
Suivre chaque machine	`prometheus-node-exporter` installé par `commun.sh` sur toutes les machines quand E2 est activée, dans le même appel apt que le socle	Paquet Ubuntu, sans téléchargement extérieur. Une machine éteinte se voit tout de suite
Suivre la répartition et la santé vues par HAProxy	L'exportateur Prometheus déjà intégré au paquet HAProxy	Aucun composant de plus : on lit directement les compteurs de réponses et l'état de la sonde de chaque serveur
Pas d'adresse écrite en dur (R3)	`prometheus.yml` est généré par `obs.sh` à partir de `plateforme.json`, vérifié par `promtool check config`, et le rôle obs est rejoué à chaque `vagrant up`	Avec `NB_APP=4`, app3 et app4 apparaissent seules dans Prometheus : le journal indique « Prometheus reconfiguré : 9 machine(s) », puis 7 au retour à deux serveurs
Un tableau de bord reproductible (R1)	La source de données et le tableau de bord sont décrits par des fichiers (`config/grafana/`), sans aucune manipulation à la souris	Un `vagrant destroy` ne fait rien perdre, puisque le tableau de bord est du code versionné
Pas de mot de passe dans le dépôt (R7)	Le mot de passe administrateur `GRAFANA_MDP` est lu dans `.env` et placé dans un fichier en mode 600	Le dépôt ne contient aucun secret
Cloisonnement (R8)	Grafana et Prometheus ne sont joignables que sur l'adresse privée de obs	Seul le répartiteur expose un port vers l'hôte

Le tableau de bord contient six panneaux : le débit par serveur, l'état de chaque serveur selon la sonde `/sante`, les machines joignables, le processeur de chaque machine, les réponses par code HTTP (2xx en vert, 5xx en rouge) et un historique de disponibilité, avec une ligne par machine, qui garde la trace des pannes provoquées pendant les scénarios. Les captures de la section 6 en sont tirées.

Nous avons rencontré deux difficultés avec Grafana. Au premier déploiement, Grafana 13 a mis 10 minutes à démarrer et le provisionnement a échoué en dépassant son délai d'attente (`tests/resultats/e2_obs_20261024_123257.log`). Son journal montrait qu'il chargeait une douzaine de sources de données intégrées (Loki, MySQL, Elasticsearch, etc.), chacune dans un processus séparé : les 768 Mo de obs ne suffisaient plus et le noyau tuait des services système. Nous avons désactivé tous les plugins sauf celui de Prometheus (`GF_PLUGINS_DISABLE_PLUGINS` et `GF_PLUGINS_PREINSTALL_DISABLED`). Le provisionnement suivant a pris 1 min 08 s, et un nouveau passage n'a rien modifié. Plus tard, lors de la validation finale du

25 octobre sur le poste de PORGO, le dépôt `apt.grafana.com` était injoignable depuis le réseau utilisé, alors que `dl.grafana.com` répondait. Depuis, `obs.sh` installe Grafana 13.2.4 à partir du paquet publié sur `dl.grafana.com`, vérifie son empreinte SHA-256 inscrite dans le script et le garde en cache dans le dossier `.cache/` du projet. La reconstruction ne dépend donc plus de ce dépôt, et elle ne télécharge plus Grafana qu'une seule fois.

3.6 Points de défaillance supprimés et points restants (groupe)

Composant	Avant	Après
Répartiteur	lb1 seul	lb1 et lb2 avec une IP virtuelle VRRP (E1)
Serveur applicatif	un seul serveur	au moins deux serveurs, sonde de santé et rejeu des requêtes
Base de données	bdd1 seule	bdd1 répliquée en flux vers bdd2, bascule chronométrée en 3 s environ et réintégration automatique de l'ancien primaire

Il reste plusieurs points faibles. La bascule de la base est manuelle : il faut lancer `bascule_bdd.sh`. Toutes les machines virtuelles tournent sur un seul ordinateur et passent par un seul réseau hôte. La machine obs est unique, mais sa perte n'interrompt pas le service.

4. Reproductibilité et qualité (PORGO)

Sur un clone neuf du dépôt, `vagrant up` suffit (R1). Si le fichier `.env` n'existe pas, il est créé à partir de `.env.exemple` avec des mots de passe aléatoires. La box est fixée à une version précise (`bento/ubuntu-24.04` en version `202510.26.0`) et tout le reste est installé par les scripts. Le premier `vagrant up` du socle a pris 1 433 s, téléchargement de la box compris. La reconstruction complète (`vagrant destroy -f && vagrant up`, avec les bases, l'application et les répartiteurs) a pris 1 934 s.

Le provisionnement peut être rejoué sans effet (R2) grâce à trois techniques. La fonction `installer_paquets` n'appelle apt que pour les paquets absents. La fonction `installer_si_change` ne remplace un fichier, et ne recharge le service concerné, que si son contenu a changé. Enfin, `/etc/hosts` est géré comme un bloc délimité par deux marqueurs et réécrit à l'identique, jamais avec un ajout `>>`. Nous l'avons vérifié en comparant les empreintes SHA-256 des fichiers avant et après un second `vagrant provision` sur les six machines (section 6).

Aucun mot de passe ne se trouve dans le dépôt (R7) : `.env` est ignoré par Git, les valeurs sont transmises aux scripts par l'option `env:` du provisionneur, et les fichiers qui les contiennent dans les machines sont en mode 600. Le fichier `.gitattributes` impose des fins de ligne Unix, car un script enregistré avec des fins de ligne Windows échouerait dans la machine virtuelle. Pour les connexions lentes, chaque script ne fait qu'un seul appel à apt, l'index n'est rafraîchi qu'une fois toutes les six heures, et l'option `DPkg::Lock::Timeout=300` permet d'attendre la fin des mises à jour automatiques au démarrage.

5. Procédures d'exploitation

5.1 Ajouter ou retirer des serveurs applicatifs (PORGO)

Pour passer à quatre serveurs, il suffit de mettre `NB_APP=4` dans `.env` (ou de lancer `NB_APP=4 vagrant up`), puis d'exécuter `vagrant up`. Vagrant crée app3 et app4, qui sont installées par `app.sh`, puis rejoue le rôle des répartiteurs. Ceux-ci régénèrent `haproxy.cfg` et rechargent HAProxy sans couper les connexions en cours. La page `http://192.168.59.11:8404/stats` affiche alors quatre serveurs actifs.

Pour revenir à deux serveurs, on détruit app3 et app4 (`NB_APP=4 vagrant destroy -f app3 app4`), on remet `NB_APP=2`, puis on relance `vagrant up`. Nous avons testé les deux sens (section 6).

5.2 Bascule de la base (OUEDRAOGO)

La bascule devient nécessaire quand bdd1, la base primaire, est tombée. Dans ce cas, `/sante` renvoie 503 sur tous les serveurs applicatifs, et HAProxy répond lui aussi en JSON (`{"erreur": "aucun serveur applicatif disponible"}`), jamais par une page blanche. Depuis la racine du dépôt, on lance :

```
time vagrant ssh bdd2 -c "sudo bash /vagrant/scripts/bascule_bdd.sh"
```

Le script commence par vérifier que l'ancien primaire est vraiment hors service, en tentant pendant deux secondes une connexion sur son port 5432. S'il répond encore, la promotion est annulée, car promouvoir bdd2 alors que bdd1 fonctionne créerait deux bases primaires. Il promeut ensuite bdd2 avec `SELECT pg_promote(true, 60)`, uniquement si elle est encore une réplique, ce qui permet de relancer le script sans effet. Il attend que `pg_is_in_recovery()` renvoie `false`, pendant 120 secondes au plus, puis affiche l'heure de début, l'heure de fin et la durée.

Rien ne change du côté de l'application : sa chaîne de connexion trouve seule le nouveau primaire. Le script fonctionne dans les deux sens : lancé sur bdd1, il promeut bdd1.

Mesure	Résultat	Fichier
Durée de la promotion	2 s (OUEDRAOGO), puis 3 s à chaque test d'intégration (PORGO)	`bdd_20261023.log`, `integration_bascule_20261023_164927.log`, `reintegration_bdd1_20261023_173638.log`
Retour du service après la fin de la bascule	5,1 s, le temps que HAProxy obtienne deux sondes réussies (`rise 2`, `downinter 2s`)	`integration_bascule_20261023_164927.log`
Commande complète vue de l'hôte	environ 10 s, dont 7 s pour ouvrir la session `vagrant ssh`	même fichier

Une fois la bascule faite, il faut remettre bdd1 en service (section 5.3).

5.3 Remettre bdd1 en service comme réplique (PORGO, pour OUEDRAOGO)

Après une bascule, bdd2 est devenue primaire. Si l'on rallume bdd1, systemd redémarre PostgreSQL en primaire, et l'on se retrouve avec deux bases primaires. Comme la chaîne de connexion des applications essaie bdd1 en premier, elles liraient l'ancienne base, à laquelle il manque tout ce qui a été écrit depuis la panne. Un deuxième problème se posait : relancer `replique.sh` sur bdd2, devenue primaire, l'aurait effacée pour la recopier depuis l'ancien primaire.

La remise en service se fait maintenant avec une seule commande, `vagrant up bdd1`. Le script `scripts/reintegrer_bdd.sh`, rejoué à chaque `vagrant up` des deux bases, s'occupe du reste. Si la machine est déjà une réplique, ou si l'autre base ne répond pas, il ne fait rien. Sinon, il compare les « lignes de temps » (timelines) de PostgreSQL : la sienne, lue avec `pg_controldata`, et celle de l'autre base, obtenue par la commande de réplication `IDENTIFY_SYSTEM`. Chaque promotion augmente ce numéro de un. Si l'autre base a un numéro plus récent, c'est que la machine courante est l'ancien primaire. Dans ce seul cas, le script arrête PostgreSQL, efface les données périmées, recopie le nouveau primaire avec `pg_basebackup -R`, redémarre en réplique et vérifie que les données arrivent bien.

Le script fonctionne aussi dans l'autre sens, si bdd2 tombe à son tour. Deux protections le complètent : `bdd.sh` n'écrit plus rien quand bdd1 est une réplique, et `replique.sh` ne réinitialise jamais bdd2 si elle contient déjà la base.

Étape	Résultat
Situation normale, réintégration lancée sur bdd1	« primaire à jour (ligne de temps 1, bdd2 : 1) » : rien n'est fait
Arrêt de bdd1, puis bascule sur bdd2	promotion en 3 s
Écriture sur bdd2 pendant que bdd1 est éteinte	ligne `REPRISE01` insérée
`vagrant up bdd1`	ancien primaire détecté (ligne de temps 1 contre 2), converti en réplique en 39 s. La commande complète dure 208 s, démarrage de la machine compris
Données	`REPRISE01` est présente sur bdd1, une nouvelle écriture sur bdd2 apparaît sur bdd1 une seconde plus tard, et les deux

Étape	Résultat
	bases comptent 50 002 lignes
Service	`/sante` répond 200 en 0,33 s, contre 2 s environ tant que bdd1 était éteinte
Provisionnement rejoué après la bascule	deux passages sur bdd1 (réplique) et sur bdd2 (primaire) : quatre succès, aucune écriture et aucune réinitialisation

Ces mesures sont dans `reintegration_bdd1_20261023_173638.log`. Il reste une limite : entre le démarrage de la machine et le passage du script, bdd1 fonctionne comme un primaire périmé pendant 42,7 s (journal systemd : démarrage à 17:41:07,3, arrêt à 17:41:50,1). Pendant ce court laps de temps, une application peut lire des données périmées, mais elle ne peut pas écrire de données divergentes puisque le service ne fait que lire. Pour supprimer cette fenêtre, il faudrait faire la même comparaison avant le démarrage de PostgreSQL, par exemple avec une condition `ExecStartPre` dans systemd.

5.4 Panne et retour d'un répartiteur (PORGO)

Si lb1 tombe, il n'y a rien à faire : lb2 prend l'IP virtuelle en moins de deux secondes. Pour le remettre en service, on lance `vagrant up lb1`. Son rôle est rejoué sans rien changer, keepalived redémarre, et lb1 reprend l'adresse dix secondes plus tard grâce à sa priorité plus élevée. On peut le vérifier avec `vagrant ssh lb1 -c "ip -4 addr show eth1"`, qui doit afficher `192.168.59.5`.

Nous avons aussi testé l'arrêt de HAProxy seul sur lb1, la machine restant allumée. Après `systemctl stop haproxy`, lb2 est passé en état MASTER 2,59 s plus tard (d'après le journal de keepalived et les horloges des deux machines), et le service est resté joignable. Au redémarrage de HAProxy, lb1 a repris l'adresse.

6. Résultats des scénarios (THIOMBIANO et responsables)

Pour chaque scénario, nous donnons la commande exacte, le fichier de résultats bruts, un tableau et notre analyse. Les scénarios V1 à V5 ont été mesurés le 24 octobre sur le poste de THIOMBIANO (Intel Core i7-1185G7, 4 cœurs et 8 threads, 15,7 Go de RAM, Windows 11 Professionnel, VirtualBox 7.2.6, Vagrant 2.4.9), avec la plateforme complète, extensions comprises (7 machines virtuelles d'un processeur virtuel chacune). V6 et les vérifications R2 à R8 ont été faites le 23 octobre sur le poste de PORGO, de configuration proche.

Scénario	Mesure attendue	Résultat	Fichiers
V1 répartition	écart inférieur à 20 %	10 et 10 appels avec deux serveurs, 5, 5, 5 et 5 avec quatre serveurs : écart de 0 %, aucun échec	`v1_20261024_140111.log`, `v1_20261024_155856.log`, `v1_20261024_161201.log`
V2 panne de app1	requêtes perdues, durée de dégradation	aucune requête perdue ni ralentie, sur trois mesures de 81 (PORGO), 139 et 132 requêtes (THIOMBIANO)	`v2_20261023_163047.log`, `v2_20261024_142807.log`, `v2_20261024_145903.log`
V3 montée en charge	requêtes par seconde	débit utile de 15,7 réponses 200 par seconde avec deux serveurs et de 12,3 avec quatre. ab annonce 26,5 et 26,2 requêtes par seconde, mais 41 % puis 53 % des réponses sont des 503 : la base limite le débit	`v3_N2_20261024_151124.log`, `v3_N4_20261024_160100.log`
V4 panne de bdd1	durée de la bascule	503 en JSON pendant la panne et aucune réponse brute sur 160 sondes. Promotion en 3 s, service rétabli 3 à 5 s après la bascule, bdd1 redevenue réplique en 25 s	`v4_20261024_161821.log`, `v4_20261024_161821_sonde.log`

Scénario	Mesure attendue	Résultat	Fichiers
V5 reconstruction	durée totale	1 778 s (29 min 38 s) pour les 7 machines, extensions comprises, sans aucune commande ensuite et sans erreur	`v5_20261024_164401.log`, `v5_20261024_164401_vagrant.log`
V6 panne de lb1	temps de reprise VRRP	avec l'application finale, une requête perdue sur 124 et 1,70 s d'interruption. Sur le poste de THIOMBIANO, une sur 114 en 1,74 s	`v6_20261023_163244.log`, `v6_20261024_182157.log`

6.1 V1 : répartition (THIOMBIANO)

La commande `bash tests/v1_repartition.sh` envoie vingt appels successifs à `http://192.168.59.5/info` et lit le nom du serveur dans l'en-tête `X-Serveur` ajouté par HAProxy. La liste des serveurs attendus vient de `plateforme.json`, si bien qu'un serveur qui ne répondrait jamais serait compté à zéro au lieu d'être oublié. L'écart correspond à la différence entre le serveur le plus sollicité et le moins sollicité, rapportée à la part attendue de chacun (20 divisé par N).

N	Répartition	Écart	Ordre observé
2	app1 : 10, app2 : 10	0 %	alternance stricte entre app2 et app1
4	app1 à app4 : 5 chacun	0 %	cycle app1, app2, app3, app4, répété cinq fois

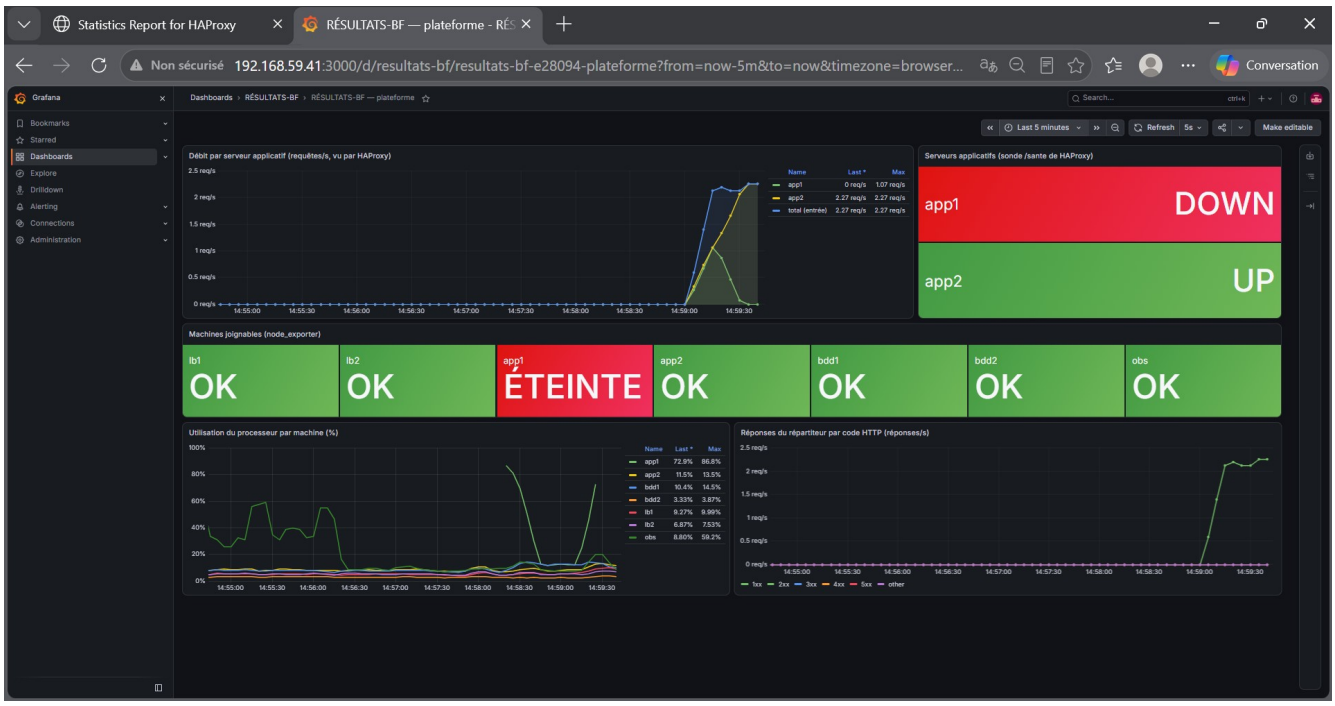
C'est le comportement attendu de la répartition à tour de rôle, avec des appels successifs et des serveurs tous en bonne santé. Un écart nul ne prouverait pas grand-chose sous une charge simultanée, mais il montre ici que tous les serveurs déclarés reçoivent du trafic, y compris app3 et app4 ajoutés par `NB_APP=4` (R3).

6.2 V2 : panne d'un serveur applicatif (KOBENDE, mesures de PORGO et THIOMBIANO)

La commande `DUREE=60 bash tests/v2_panne_app.sh` envoie une requête toutes les 200 ms sur l'IP virtuelle et arrête app1 avec `vagrant halt` au bout de 5 secondes.

Mesure	Requêtes	Perdues	Ralenties (plus de 0,5 s)	app1 répond encore après l'ordre d'arrêt pendant
23/10 (PORGO)	81	0	0	13,1 s
24/10 à 14 h 28	139	0	0	13,2 s
24/10 à 14 h 59	132	0	0	9,6 s

Dans les trois mesures, la panne passe inaperçue pour les utilisateurs. `vagrant halt` éteint app1 proprement et elle continue de répondre pendant l'arrêt. Dès qu'elle cesse de répondre, la sonde l'écarte, et les requêtes déjà envoyées vers elle sont rejouées sur app2. Sur la capture C3, app1 apparaît hors service et éteinte, app2 prend tout le débit et aucune réponse 5xx n'apparaît.



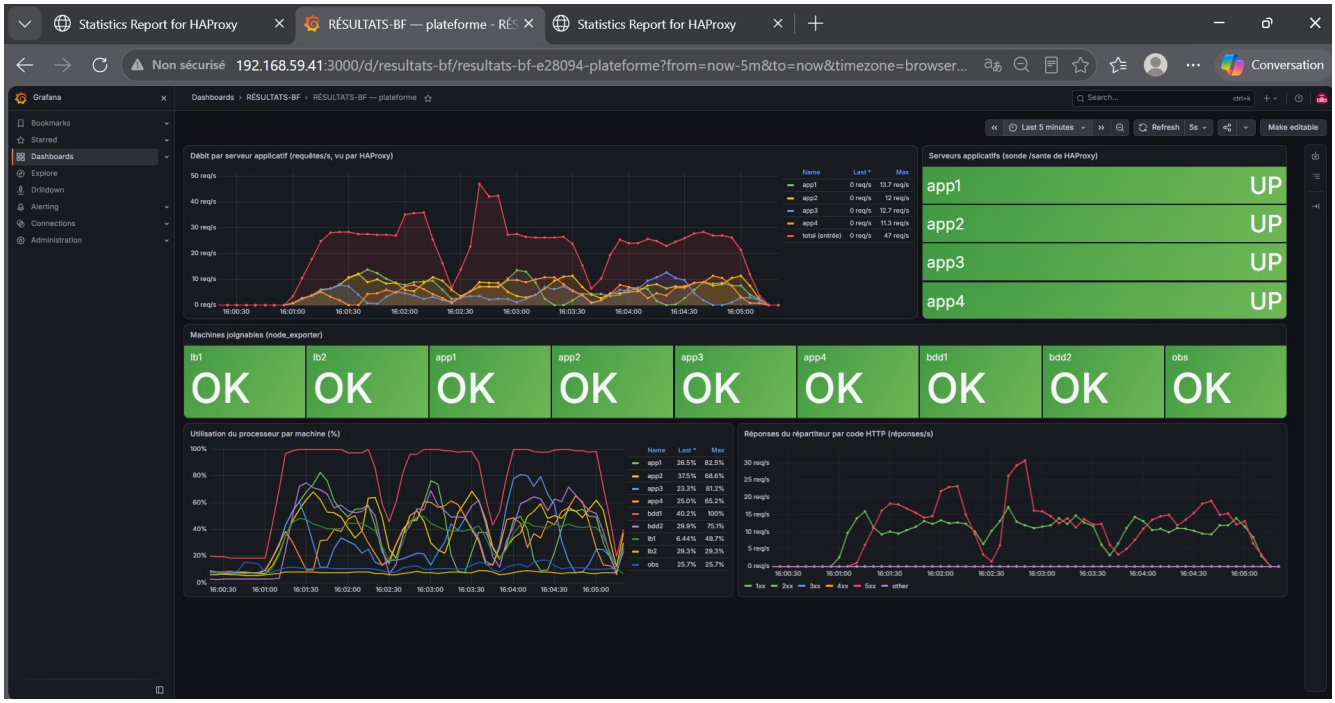
C3 : le scénario V2 vu dans Grafana. app1 est écartée et éteinte, app2 reprend tout le débit, et seules des réponses 2xx apparaissent.

6.3 V3 : montée en charge avec deux puis quatre serveurs (THIOMBIANO)

La commande `bash tests/v3_montee_charge.sh` exécute trois tirs de `ab -l -n 2000 -c 50 http://192.168.59.5/resultat?matricule=BF000123`, après un tir d'échauffement non compté et avec 10 secondes de pause entre deux tirs. Nous retenons le tir médian. Entre les deux séries, nous avons mis `NB_APP=4` dans `.env` et relancé `vagrant up`, qui a créé app3 et app4 et reconfiguré HAProxy et Prometheus en 4 min 22 s. Comme ab compte aussi les réponses 503 dans son nombre de requêtes par seconde, le script calcule également le débit utile, c'est-à-dire le nombre de réponses 200 par seconde.

Tir	N = 2 : req/s (ab)	N = 2 : réponses 200/s	N = 2 : réponses non 2xx	N = 4 : req/s (ab)	N = 4 : réponses 200/s	N = 4 : réponses non 2xx
1	25,91	14,78	43,0 %	28,96	11,86	59,0 %
2	26,49	15,68	40,8 %	30,32	13,23	56,4 %
3	37,35	17,42	53,4 %	26,25	12,35	53,0 %
Médiane	26,5	15,7	40,8 %	26,2	12,3	53,0 %
Temps de réponse médian	1,5 à 1,9 s			1,7 à 1,8 s		

Le débit n'augmente pas quand on passe à quatre serveurs : le débit utile baisse même de 22 %. Plusieurs éléments l'expliquent. D'abord, c'est la base qui limite, pas les serveurs applicatifs : pendant chaque tir, bdd1, qui n'a qu'un processeur virtuel, est à 100 % d'utilisation, alors que les serveurs applicatifs restent entre 50 et 90 % (captures C4 et C5). Ensuite, chaque requête ouvre une nouvelle connexion PostgreSQL, puisque nous avons choisi une application sans état et sans réserve de connexions (section 3.2). Chaque connexion lance un nouveau processus sur le serveur et une authentification SCRAM-SHA-256 de 4 096 itérations, ce qui coûte bien plus cher que la lecture d'une ligne par sa clé primaire. Avec 50 requêtes simultanées, des connexions attendent plus de deux secondes et atteignent le `connect_timeout` : l'application renvoie alors un 503 au lieu de faire attendre l'utilisateur. Les journaux de PostgreSQL ne contiennent aucune erreur, ce sont donc bien des attentes qui expirent. Sous cette charge, `/sante` dépasse parfois lui aussi son délai, si bien que HAProxy écarte puis réintègre des serveurs quelques secondes plus tard : après la série à deux serveurs, les compteurs indiquaient 16 exclusions pour app1 (dont deux dues aux arrêts volontaires de V2) et 6 pour app2. Enfin, avec quatre serveurs, deux fois plus de connexions simultanées arrivent sur la même base saturée, et neuf machines virtuelles se partagent les quatre cœurs du même ordinateur.



C5 : le scénario V3 avec quatre serveurs. On voit les trois tirs, les quatre serveurs au même débit, bdd1 à 100 % de processeur (courbe rouge) et les réponses 5xx qui accompagnent chaque tir.

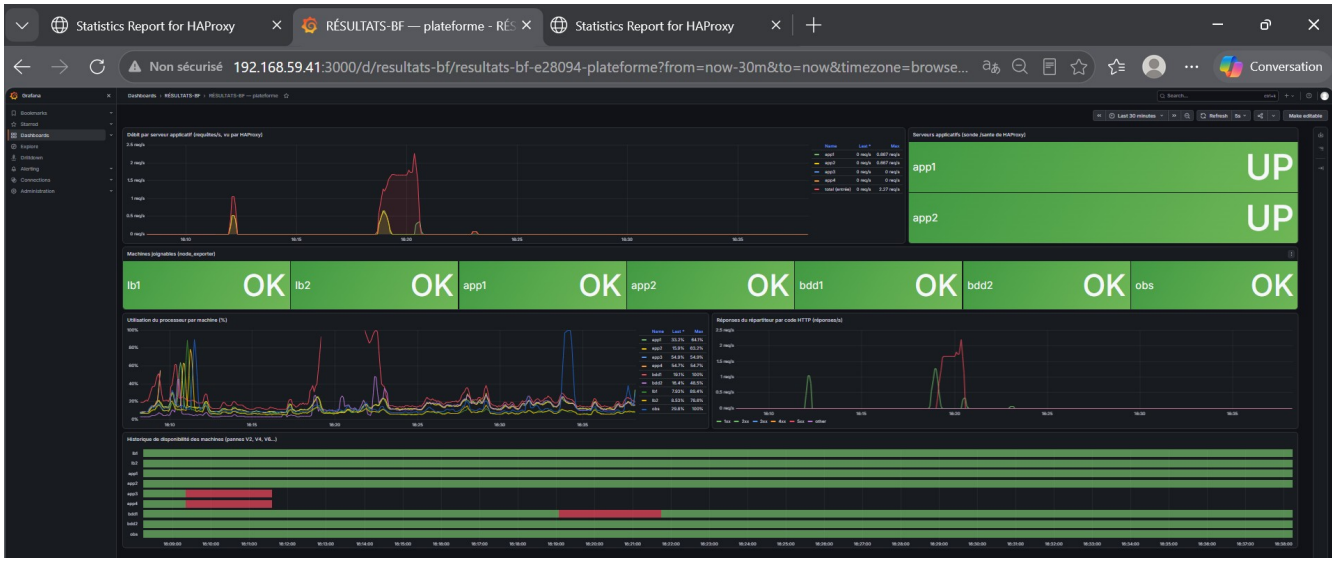
La plateforme garde malgré tout un comportement correct sous cette charge : aucune requête ne reste sans réponse (ab indique 0 requête en échec sur 12 000) et aucune page d'erreur brute n'est renvoyée. Le service se dégrade progressivement au lieu de s'effondrer. Pour que l'ajout de serveurs augmente vraiment le débit, il faudrait une réserve de connexions comme PgBouncer, afin que la base ne paie plus une authentification par requête, puis répartir les lectures sur la réplique bdd2 et installer les machines sur plusieurs hôtes (section 7).

6.4 V4 : panne de la base primaire et bascule (OUEDRAOGO, mesure de THIOMBIANO)

La commande `bash tests/v4_panne_bdd.sh` interroge `/sante` sur l'IP virtuelle toutes les 500 ms pendant tout le scénario. Elle arrête bdd1 avec `vagrant halt`, lance la bascule sur bdd2, attend le retour à 200, puis remet bdd1 en service avec `vagrant up bdd1`. Les instants sont comptés à partir de l'ordre d'arrêt.

Étape	Résultat
Avant	bdd1 primaire, bdd2 réplique, `/sante` répond 200
18,1 s : premier 503	PostgreSQL s'arrête au début de l'extinction de la machine
Pendant la panne	`/sante` et `/resultat` répondent `{"erreur": "aucun serveur applicatif disponible"}` avec le code 503, en JSON
78,7 s	la machine bdd1 est éteinte (l'arrêt propre a duré 78 s)
Bascule	promotion en 3 s, commande complète de 12 s avec l'ouverture de la session `vagrant ssh`
Retour du service	3,2 s (d'après la sonde) à 5,0 s (d'après la boucle d'attente) après la fin de la bascule, sans redémarrer l'application
Sondes	160 au total : 30 réponses 200, 130 réponses 503 et aucune réponse sans code HTTP
Interruption vue par les utilisateurs	79,1 s entre le premier 503 et le retour à 200
Remise en service avec `vagrant up bdd1`	129 s, dont 25 s de conversion en réplique après la détection de l'ancien primaire (ligne de temps 1 contre 2)
Après	bdd1 réplique, bdd2 primaire, `/sante` répond 200

L'exigence est respectée : le service renvoie un 503 propre pendant la panne, jamais une erreur brute, et la procédure de bascule est exécutable et chronométrée. Il faut toutefois lire les 79 secondes d'interruption avec prudence, car elles viennent surtout de la façon dont nous provoquons la panne. Le script attend que `vagrant halt` ait fini d'éteindre la machine, soit 60 secondes après l'arrêt de PostgreSQL, avant de lancer la bascule, qui refuse de promouvoir bdd2 tant que l'ancien primaire répond. La procédure elle-même coûte 12 secondes de commande, puis 3 à 5 secondes pour que HAProxy réintègre les serveurs. Une bascule lancée dès le premier 503 aurait donc ramené l'interruption à une vingtaine de secondes, plus le temps de réaction d'un opérateur. Il s'agit là d'une estimation et non d'une mesure. Pour supprimer cette attente, il faudrait une bascule automatique (section 7).



C6b : historique de disponibilité. Le bloc rouge de bdd1, entre 16 h 19 et 16 h 22, correspond à la panne V4. app3 et app4 disparaissent au retour à deux serveurs.

6.5 V5 : reconstruction complète (THIOMBIANO)

La commande `bash tests/v5_reconstruction.sh` exécute `vagrant destroy -f` puis `vagrant up` sans aucune intervention, et vérifie ensuite la plateforme. Les paramètres étaient `GROUPE=4`, `NB_APP=2`, `E1_LB2=1` et `E2_OBS=1` (7 machines), avec la box déjà présente sur le poste.

Phase	Durée
`vagrant destroy -f` sur les 7 machines	36 s
`vagrant up` (bases, réplication, application, lb1, lb2, obs)	1 742 s, sans erreur
Total	1 778 s, soit 29 min 38 s
Vérification	7 machines en marche, `/sante` et `/resultat?matricule=BF000123` répondent 200, aucune erreur dans le journal de Vagrant

R1 est donc respectée sur la plateforme complète, extensions comprises, et sur un autre poste que celui où elle a été développée. La durée vient surtout des téléchargements apt (PostgreSQL, HAProxy, Grafana) et du démarrage des machines les unes après les autres. Elle reste du même ordre que les 1 934 s mesurées sur le poste de PORGO sans obs. Le 25 octobre, une dernière reconstruction complète des 7 machines sur le poste de PORGO, avec la version finale du code et Grafana installé depuis son cache, a également abouti sans erreur.

6.6 V6 : panne du répartiteur (PORGO)

La commande `DUREE=45 bash tests/v6_panne_lb.sh` envoie une requête toutes les 200 ms vers `http://192.168.59.5/info` et arrête lb1 avec `vagrant halt` au bout de 5 secondes. `vagrant halt` éteint lb1 proprement en une dizaine de secondes, pendant lesquelles il continue de servir. La seule requête perdue tombe au moment où keepalived s'arrête sur lb1 et où lb2 n'a pas encore pris l'adresse. L'interruption mesurée, de 1,7 s environ, reste sous les 5 secondes demandées. Elle s'explique par le fait que keepalived, en s'arrêtant, envoie une annonce de priorité nulle : lb2 n'a donc pas besoin

d'attendre l'expiration des annonces. En cas de coupure brutale, comme une panne de courant, lb2 attendrait environ trois à quatre secondes, ce qui resterait sous la limite.

Nous avons aussi vérifié les exigences de répartition avec `bash tests/verifier_repartition.sh` :

Exigence	Méthode	Résultat	Fichier
R1	`vagrant destroy -f && vagrant up` de la plateforme sans obs, sans commande ensuite	réussite en 1 934 s, box déjà présente. Le premier `vagrant up` du socle avait pris 1 433 s avec le téléchargement de la box	journal de la console
R1 sur un deuxième poste	clone neuf sur le poste de THIOMBIANO, puis `vagrant up`	réussite en 1 560 s, téléchargement de la box compris	`r1_poste_thiombiano_20261024_111253.log`
R2	deux `vagrant provision` successifs sur les 6 machines, avec comparaison des empreintes des fichiers de configuration et des numéros de processus des services	aucune différence sur les 6 machines	`repartition_20261023_162550.log`
R3	passage de 2 à 4 serveurs puis retour à 2	4 min 22 s. HAProxy est reconfiguré avec 4 serveurs sur lb1 et lb2, Prometheus avec 9 machines, puis tout revient à 2 serveurs et 7 machines. Les 20 appels se répartissent 5, 5, 5 et 5	`r3_nb_app4_20261024_152310.log`, `repartition_20261023_110244.log`
R4	arrêt du service sur app1 et lecture des statistiques toutes les 200 ms	app1 écartée en 1 394 ms, puis en 1 279 ms avec le délai de lecture de 3 s, pour une exigence de 5 000 ms	`repartition_20261023_162950.log`, `repartition_20261023_165028.log`
R5	`bash tests/verifier_replication.sh` (OUEDRAOGO) : insertion d'une ligne datée sur bdd1, lecture sur bdd2, puis suppression	la ligne est lisible sur bdd2 dès la première lecture, en une seconde au plus, sur le poste d'OUEDRAOGO comme sur celui de PORGO	`r5_replication_20261025.log`, `r5_replication_20261025_141141_poste_porgo.log`
R8	`vagrant port` sur chaque machine	seul lb1 redirige un port (80 vers 8080), les autres n'ont que l'accès SSH de Vagrant	`repartition_20261023_104842.log`

Les premières mesures, le matin du 23 octobre, ont été faites avec une petite application de test qui répondait 200 sur toutes les routes. Nous avons refait R1, R2, R4, V2 et V6 l'après-midi avec l'application finale et la base répliquée : ce sont ces chiffres qui figurent ci-dessus. Nous avons aussi vérifié les réponses de l'application finale à travers l'IP virtuelle : 200 pour un matricule existant (y compris `BF050000`), 400 sans paramètre, 404 pour un matricule ou une route inconnus. Une tentative d'injection SQL (`BF000001' OR 1=1--`) renvoie simplement 404.

Le poste de PORGO est équipé d'un processeur Intel Core i7-1165G7 (4 cœurs, 8 threads) et de 15,7 Go de RAM, sous Windows 11 Professionnel, avec VirtualBox 7.2.6 et Vagrant 2.4.9. Les machines virtuelles tournent sous Ubuntu 24.04.3 LTS (noyau 6.8), avec HAProxy 2.8.30 et keepalived 2.2.8.

7. Limites connues et ce que nous ferions autrement (groupe)

Limite	Conséquence	Ce que nous ferions autrement
Toutes les machines virtuelles partagent le processeur d'un seul ordinateur	Ajouter des serveurs ne multiplie pas le débit (V3) : les tests de charge mesurent surtout ce partage	Répartir les machines sur plusieurs hôtes, ou mesurer dans un vrai cloud
La base primaire limite le débit (un processeur virtuel, une connexion authentifiée par requête)	Pendant V3, bdd1 est à 100 % et, avec 50 clients simultanés, 41 % puis 53 % des réponses sont des 503. Le débit utile baisse de 15,7 à 12,3 réponses par seconde quand on double le nombre de serveurs	Placer PgBouncer devant la base, répartir les lectures sur la réplique et donner plus de processeurs à la base
La bascule de la base est manuelle	Le service reste en 503 jusqu'à ce que quelqu'un lance `bascule_bdd.sh`	Automatiser la bascule avec Patroni et etcd, qui évitent aussi d'avoir deux primaires
La réplication est asynchrone	Les dernières transactions pourraient être perdues si l'application écrivait	Passer en réplication synchrone, avec une troisième base pour ne pas bloquer les écritures
bdd1 reste primaire pendant 42,7 s quand elle revient après une bascule (section 5.3)	Des lectures périmées sont possibles pendant ce temps	Une condition `ExecStartPre` sur PostgreSQL, ou Patroni
Un serveur applicatif figé n'est écarté qu'au bout de 7 s environ (section 3.1)	Cela dépasse les 5 s pour ce cas précis, alors que les pannes franches restent autour de 1,3 s	Une sonde `/sante` plus rapide, qui n'essaie que les bases joignables
La box `bento/ubuntu-24.04` est prévue pour les processeurs Intel et AMD	La plateforme ne tourne pas sur un Mac à processeur ARM, ce qui était le cas d'un membre du groupe	Prévoir une box arm64 et un hyperviseur adapté, comme VMware Fusion ou Parallels
L'interruption de V4 dépend surtout de l'extinction de bdd1	79 s mesurées, dont 60 s d'arrêt de la machine avant la bascule	Une bascule automatique déclenchée dès la perte du primaire
Grafana 13 est lourd pour 768 Mo de mémoire	Premier démarrage de 10 minutes avant correction (section 3.5), et 530 Mo utilisés sur 709	Donner 1 Go à obs, ou choisir un outil de visualisation plus léger

8. Annexes

8.1 Organisation du dépôt

```
Vagrantfile                  table NOEUDS, source unique de vérité
.env.exemple                 modèle de configuration (le .env n'est jamais versionné)
app/app.py                   service HTTP sans état (87 lignes)
scripts/lib.sh               fonctions communes (apt groupé, idempotence, attente)
scripts/plateforme.py        lecture de config/plateforme.json
scripts/commun.sh            socle installé sur toutes les machines
scripts/lb.sh                HAProxy et keepalived
scripts/generer_config_lb.py génération de haproxy.cfg et keepalived.conf
scripts/app.sh               déploiement du service applicatif
scripts/bdd.sh               PostgreSQL primaire et jeu de données
scripts/replique.sh          réplique en flux
scripts/bascule_bdd.sh       promotion chronométrée
scripts/reintegrer_bdd.sh    remise en service de l'ancien primaire
scripts/obs.sh               Prometheus et Grafana (E2)
config/grafana/              source de données et tableau de bord
tests/                       scénarios V1 à V6 et vérifications R2 à R8
tests/resultats/             résultats bruts horodatés cités dans ce dossier
docs/captures/               captures citées en section 6
docs/                        installation, plan de travail, démonstration, dossier et journal
```

8.2 Extrait de la configuration HAProxy générée avec deux serveurs

```
defaults
    retries 2
    retry-on conn-failure empty-response response-timeout
    timeout connect 1s
    timeout check   3s
    default-server inter 1s fastinter 500ms downinter 2s fall 2 rise 2

backend serveurs_applicatifs
    balance roundrobin
    option httpchk
    http-check send meth GET uri /sante
    http-check expect status 200
    server app1 192.168.59.21:3000 check
    server app2 192.168.59.22:3000 check
```

Avec `NB_APP=4`, deux lignes `server app3` et `server app4` s'ajoutent automatiquement, sans autre modification.